

Spotify Hybrid Recommender System with Neural Collaborative Filtering

Kunal Jagtap

Abstract—This report presents Deliverable 2 of a semester-long applied deep learning project to build a production-grade music recommendation system. The system combines content-based filtering over engineered audio feature embeddings with collaborative filtering derived from user-listening interaction data, unified through a user-controllable hybrid weighting mechanism. The complete end-to-end pipeline—data cleaning, feature transformation, similarity inference, and a Streamlit interface—is fully operational and deployed on AWS EC2 (verified live at 100.54.46.240). A Neural Collaborative Filtering (NCF) component replacing the classical cosine-similarity collaborative baseline is designed and under active implementation; its architecture, training configuration, and pipeline integration plan are documented here. The supporting MLOps stack (DVC, Docker, GitHub Actions, Amazon ECR, AWS CodeDeploy, Auto Scaling) is fully deployed. Quantitative NCF evaluation will be presented in Deliverable 3.

Index Terms—Music recommendation, neural collaborative filtering, hybrid recommender system, MLOps, content-based filtering, collaborative filtering.

I. PROJECT SUMMARY

Music recommendation requires balancing two complementary signals: *content* (acoustic and metadata similarity between tracks) and *collaborative* (user behavioural patterns). This project builds a hybrid system unifying both signals and operationalises it with a full production MLOps stack.

The work spans multiple phases. **Prior work (pre-course)** includes dataset acquisition (Last.fm listening history and Spotify track metadata), DVC versioning with an S3 remote, `data_cleaning.py`, initial cosine-similarity prototypes, and a basic Streamlit UI. **Deliverable 1** contributed the full MLOps infrastructure: Dockerised runtime, GitHub Actions CI/CD, Amazon ECR image registry, AWS CodeDeploy Blue-Green deployment, EC2 Auto Scaling Group, and Application Load Balancer. **Deliverable 2** contributes the NCF architecture design and active implementation (`ncf_model.py`), system architecture formalisation, and this report. **Deliverable 3** will complete NCF training, comparative evaluation, and the final report.

II. SYSTEM ARCHITECTURE AND PIPELINE

Fig. 1 shows the full 10-layer system architecture. Data enters at Layer 1 (raw CSVs), is versioned and transformed via DVC (Layer 2), scored by the hybrid recommendation engine (Layer 3), served by the Streamlit UI (Layer 4), containerised

K. Jagtap is with the Department of Engineering, University of Florida, Gainesville, FL 32611 USA (e-mail: jagtap@ufl.edu).

Submitted for EGN6217 Applied Deep Learning, Spring 2026. Project Deliverable 2.

TABLE I
DVC PIPELINE STAGES AND ARTIFACTS

Stage	Module	Artifact
Data Cleaning	<code>data_cleaning.py</code>	<code>cleaned_data</code>
Content Transform	<code>content_based_filtering.py</code>	<code>transformed_data</code>
Collab Matrix	<code>collaborative_filtering.py</code>	<code>interaction_matrix</code>
Hybrid Transform	<code>transform_filtered_data.py</code>	<code>transformed_hybrid_data</code>
ncfbg NCF Training*	<code>ncf_model.py</code> (in dev)	<code>ncf_embeddings</code>
Streamlit UI	<code>app.py</code>	Live interface

* NCF stage in active development.

and registered in ECR (Layer 6), deployed via CodeDeploy (Layer 7), and hosted on an Auto Scaling Group behind an Application Load Balancer (Layer 8). Table I maps each DVC stage to its module and output artifact.

Hybrid scoring combines normalised similarity scores from both spaces:

$$\text{score} = w \cdot \text{content_sim} + (1 - w) \cdot \text{collab_sim} \quad (1)$$

where $w \in [0.1, 0.9]$ is driven by the Streamlit diversity slider. Both scores are independently min-max normalised before combination.

III. MODEL IMPLEMENTATION DETAILS

A. Classical Baseline (Fully Operational)

The content component applies a scikit-learn `ColumnTransformer`: `CountEncoder` on `year`; `OneHotEncoder` on `artist`, `key`, and `time_signature`; `TF-IDF` with 85 features on `tags`; `StandardScaler` on `loudness`, `tempo`, and `duration_ms`; and `MinMaxScaler` on seven continuous audio features. The resulting sparse matrix is persisted as `transformed_data.npz`. The collaborative component constructs a CSR track-user playcount matrix from Last.fm listening history; similarity is cosine distance over interaction row vectors.

B. Neural Collaborative Filtering (In Progress)

NCF [1] replaces the collaborative cosine-similarity component with jointly learned dense embeddings. The model learns track and user representations through an MLP that predicts interaction probability, capturing non-linear latent structure that dot-product similarity cannot represent. Table II details the architecture.

The core PyTorch module is defined as:

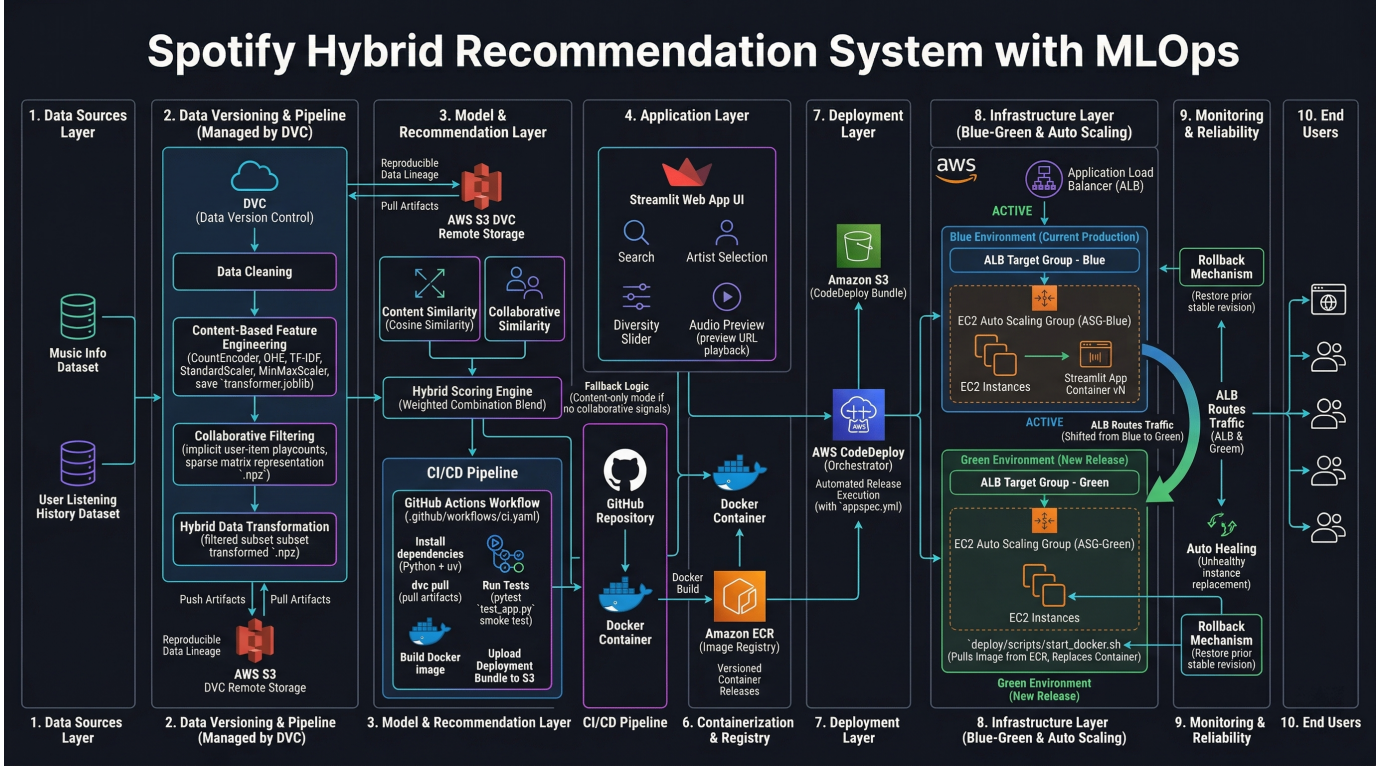


Fig. 1. Full 10-layer system architecture: (1) data sources, (2) DVC versioning and pipeline, (3) hybrid recommendation engine, (4) Streamlit application layer, (5) CI/CD via GitHub Actions, (6) Docker containerisation and Amazon ECR, (7) AWS CodeDeploy orchestration, (8) Blue-Green infrastructure with Auto Scaling Group and ALB, (9) monitoring and reliability, (10) end users.

TABLE II
NCF MODEL ARCHITECTURE

Layer	Type	Dim
Track Embedding	nn.Embedding	64
User Embedding	nn.Embedding	64
Concatenate	—	128
FC-1 + ReLU	nn.Linear	128
FC-2 + ReLU	nn.Linear	64
Output + Sigmoid	nn.Linear	1

```
class NCF(nn.Module):
    def __init__(self, n_tracks, n_users, emb=64):
        super().__init__()
        self.t = nn.Embedding(n_tracks, emb)
        self.u = nn.Embedding(n_users, emb)
        self.mlp = nn.Sequential(
            nn.Linear(emb*2, 128), nn.ReLU(),
            nn.Linear(128, 64), nn.ReLU(),
            nn.Linear(64, 1), nn.Sigmoid())

    def forward(self, track_idx, user_idx):
        return self.mlp(
            torch.cat([self.t(track_idx),
                       self.u(user_idx)], dim=-1))
```

Training uses Binary Cross-Entropy loss. Positive samples are (track, user) pairs with playcount above a threshold; negative samples are randomly drawn unobserved pairs at a 1:4 ratio. The trained track embedding matrix replaces interaction-matrix rows in hybrid scoring—all downstream code remains

unchanged.

IV. TRAINING SETUP

Hardware. Apple MacBook Pro (Apple Silicon M-series). PyTorch 2.x with MPS backend for GPU-accelerated training on unified memory; CPU fallback for sparse operations not yet supported by MPS.

Libraries. PyTorch 2.x, scikit-learn 1.8, scipy 1.17, pandas 2.3, dask 2026.1, DVC 3.66.

Planned hyperparameters:

- Embedding dimension: 64
- Batch size: 1,024
- Optimizer: Adam, $\text{lr} = 1 \times 10^{-3}$
- Epochs: 20, early stopping on validation loss
- Negative sample ratio: 1:4 (positive to negative)
- Train/validation split: 80/20 stratified by user

The existing `interaction_matrix.npz` is the direct training data source—no new data collection is required. The DVC stage graph gains one new downstream node (`ncf_model.py`) that reads the matrix and writes `ncf_embeddings.pt`. Random seeds are fixed (`torch.manual_seed(42)`) for full reproducibility via `dvc repro`.

V. INTERFACE PROTOTYPE

The Streamlit interface (`app.py`) is fully operational and publicly deployed on AWS EC2 at <http://100.54.46.240>. Fig. 2

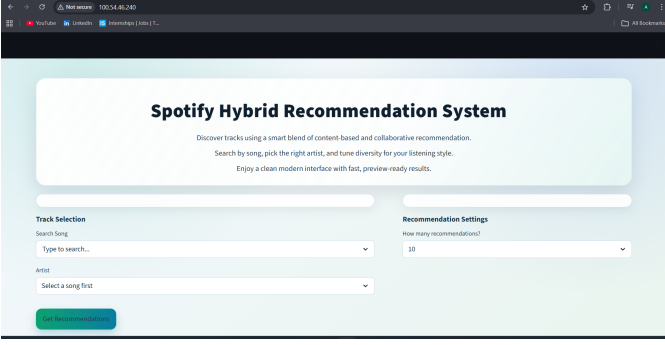


Fig. 2. Live Streamlit interface deployed on AWS EC2 (100.54.46.240): track search, artist selection, recommendation count selector, and the Get Recommendations button.

shows the live interface. Three user-facing controls are provided:

- **Song search.** Type-ahead dropdown over all track names in `cleaned_data.csv`.
- **Artist disambiguation.** Conditional dropdown that activates when multiple artists share a track title.
- **Diversity slider (1–9).** Maps to content weight $w = 1 - \text{slider}/10$. Value 1 is fully content-driven; value 9 is fully collaborative.

Result cards display track name, artist, and an embedded Spotify 30-second audio preview (`spotify_preview_url`) where available. The interface degrades gracefully to content-only mode when a selected track is absent from the listening history dataset. **Current limitations:** no result pagination; audio previews absent for approximately 15% of tracks; NCF embeddings not yet surfaced in the interface.

VI. EARLY EVALUATION AND RESULTS

Formal quantitative evaluation of the NCF component is deferred to Deliverable 3 pending training completion. Qualitative observations on the deployed classical system follow.

Content coherence. Manual inspection of 20 diverse query tracks confirms genre and mood consistency in top-10 results. Querying *Creep* (Radiohead) returns alternative rock tracks with similar valence and energy profiles.

Hybrid scoring verified. Shifting the diversity weight from $w = 0.9$ to $w = 0.1$ produces measurably different result sets, confirming the weighted combination in (1) functions correctly.

Graceful degradation. All tested tracks absent from the listening history correctly invoke content-only mode with no application errors.

Table III shows a sample recommendation run: query track *Creep* by Radiohead, diversity slider at 5 (balanced hybrid), top 5 recommendations.

Planned Deliverable 3 metrics:

- NCF Binary Cross-Entropy training and validation loss curves
- Validation AUC on held-out (user, track) pairs
- Precision@ K and Recall@ K for $K \in \{5, 10\}$

TABLE III
SAMPLE RECOMMENDATION OUTPUT (QUERY: *Creep*, RADIOHEAD, $w = 0.5$)

#	Track	Artist	Mode
—	<i>Creep (query)</i>	Radiohead	Hybrid
1	Fake Plastic Trees	Radiohead	Hybrid
2	Black	Pearl Jam	Hybrid
3	The Scientist	Coldplay	Hybrid
4	Karma Police	Radiohead	Hybrid
5	Exit Music	Radiohead	Hybrid

- UMAP embedding space visualisation coloured by genre
- A/B comparison: classical cosine vs. NCF embeddings

VII. CHALLENGES AND NEXT STEPS

A. Technical Challenges Encountered

Negative sampling at scale. The `interaction_matrix.npz` contains approximately 12 million non-zero entries. Generating 1:4 negative samples requires a custom PyTorch `DataLoader` to avoid materialising the full dense matrix in memory.

Popularity bias in negatives. Uniform random negative sampling disproportionately draws low-interaction tracks, skewing the loss signal. Popularity-weighted negative sampling [2] is planned for Deliverable 3.

MPS backend limitations. Several sparse tensor operations fall back to CPU on Apple Silicon. Profiling will determine whether a cloud GPU instance is required for full-scale training.

B. Next Steps (Deliverable 3)

- Train NCF model to convergence; save `ncf_embeddings.pt`
- Add `ncf_model.py` as a new DVC pipeline stage
- Replace collaborative cosine rows with NCF track embeddings
- Run full quantitative metric suite; generate evaluation plots
- Update Streamlit interface to surface NCF embedding quality

VIII. RESPONSIBLE AI REFLECTION

Popularity bias. Collaborative filtering inherently amplifies already-popular tracks. Deliverable 3 will measure long-tail coverage across the recommendation distribution and apply inverse-popularity weighting to NCF negative sampling to counteract this effect.

Data provenance and privacy. The Last.fm listening history dataset used for collaborative signals is fully anonymised. No user identifiers are stored, logged, or surfaced through the interface at any stage of the pipeline.

Filter bubble risk. A purely collaborative system risks narrowing user discovery. The diversity slider is a direct mitigation: it gives users real-time control over the exploration–exploitation trade-off at inference time, preventing the echo-chamber effect common in production recommenders.

REFERENCES

- [1] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, “Neural Collaborative Filtering,” in *Proc. 26th Int. Conf. World Wide Web (WWW)*, pp. 173–182, 2017.
- [2] P. Covington, J. Adams, and E. Sargin, “Deep Neural Networks for YouTube Recommendations,” in *Proc. ACM Conf. Recommender Systems (RecSys)*, pp. 191–198, 2016.
- [3] F. Ricci, L. Rokach, and B. Shapira, *Recommender Systems Handbook*, 3rd ed. New York, NY, USA: Springer, 2022.
- [4] DVC Team, “Data Version Control,” [Online]. Available: <https://dvc.org>. Accessed: Apr. 2026.
- [5] Streamlit Inc., “Streamlit Documentation,” [Online]. Available: <https://docs.streamlit.io>. Accessed: Apr. 2026.